7-1 · LangChain 챗봇 기초 · 이론 1H

LLM API와 LangChain 아키텍처

LLM 을 부르는 법과 LangChain 의 역할

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① LLM API 호출 구조와 LangChain 의 역할을 설명할 수 있다

② LLM 의 세 가지 한계와 RAG 가 푸는 방법을 말할 수 있다

③ ChatModel · 템플릿 · 파서 · 체인 네 부품의 역할을 구별한다

④ 대화 기억이 모델이 아니라 이력 재전송에서 온다는 것을 안다

다음 1.5H 실습에서 모델이 모르는 것을 일부러 물어 환각을 직접 확인합니다

1. 이번 주의 지도

오늘은 「LLM 답변 생성」 조각만 — 금요일에 전체를 잇는다

7-1 오늘	7-2	7-3	7-4	7-5
LLM 호출과 대화 우리 데이터 없음	문서를 잘라 청크로	청크를 벡터로 Qdrant 에 저장	MinIO 문서 → 임베딩 → Qdrant	키워드 + 벡터 하이브리드 검색

6주차까지 모으고 옮기고 저장한 데이터를, 이번 주에는 사람이 말로 물어볼 수 있게 만든다

오늘은 아직 우리 데이터를 쓰지 않는다 — 모델이 아는 것만으로 답하는 상태

「다음에 올 말」을 예측하는 모델 — 그리고 세 가지 한계

1. **아주 많은 글을 읽고 다음 말을 예측**
   우리는 API 로 프롬프트(질문)를 보내고 생성된 답을 받는다
2. **한계 ① 학습 시점**
   학습한 시점까지의 내용만 안다 — 그 뒤의 일이나 우리 회사 내부 문서는 모른다
3. **한계 ② 환각**
   모르는 것을 「모른다」고 하지 않고 그럴듯하게 지어내는 경향
4. **한계 ③ 컨텍스트 창**
   한 번에 처리할 수 있는 글의 양에 한계가 있다
5. **이 셋을 푸는 방법이 RAG**
   답할 근거를 찾아서 함께 넣어 주는 것 — 이번 주 나머지 나흘의 일

파인튜닝과 RAG — 다른 문제를 푼다

파인튜닝

모델을 우리 데이터로 학습시킨다

- 모델 자체를 다시 학습
- 말투·형식을 바꾸는 것이 목적일 때
- 이번 주에는 다루지 않는다

VS

RAG

답할 때 근거를 찾아 넣어 준다

- 모델은 그대로, 프롬프트에 근거를 얹는다
- 문서가 자주 바뀌거나 출처를 밝혀야 할 때
- 이번 주가 다루는 방법

3. LangChain 이 대신해 주는 것

어댑터이자 조립 블록 — prompt | llm | parser

PromptTemplate	ChatModel	OutputParser
질문의 틀 {question} 빈칸	LLM 호출을 감싼 객체 invoke() 한 가지	응답 객체에서 원하는 형태만

LCEL 체인 — 컨베이어 벨트처럼 앞의 출력이 뒤의 입력이 된다

여행용 멀티어댑터처럼 OpenAI 든 Ollama 든 같은 코드 모양(invoke)으로 부른다 — 파이프(|)는 1-1 셸 파이프와 같은 발상

3. LangChain 이 대신해 주는 것

부품 다섯 개 — 하는 일과 코드 모양

부품	하는 일	코드 모양
ChatModel	LLM 호출을 감싼 객체	어떤 모델이든 invoke()
PromptTemplate	질문의 틀	{question} 빈칸을 실행 때 채움
OutputParser	원하는 형태만 꺼내기	응답 객체 → 문자열
LCEL 체인	부품을 파이프로 잇기	prompt | llm | parser
메시지 이력	멀티턴의 기억	메시지 목록을 매번 함께 보냄

실습 파일은 모두 rag_env.py 의 llm 을 불러 쓴다 — OpenAI 든 Ollama 든 경로는 그 한 파일에서만 갈린다

4. 메시지의 세 종류

System · Human · AI — 누가 말하는가

종류	누가 말하는가	쓰임
SystemMessage	개발자가 모델에게	역할·말투·규칙을 정한다
HumanMessage	사용자	실제 질문
AIMessage	모델	모델의 답 — 이력에 넣어 문맥을 잇는다

대화가 이어지는 것처럼 보이는 것은 매번 이 목록 전체를 다시 보내기 때문 — 모델 쪽에 기억이 남는 것이 아니다

토큰 수로 과금 — 이력이 길수록 비용이 는다

1. **토큰은 대략 단어 조각**
   API 는 대개 토큰 수로 과금 — 한국어는 영어보다 토큰을 많이 쓴다

2. **이력은 매 호출마다 전체 재전송**
   비용이 대화 길이에 비례해 늘어난다

3. **그래서 잘라내거나 요약한다**
   오래된 이력을 잘라내거나 요약해 넣는 기법을 쓴다

4. **temperature = 답의 무작위성**
   0이면 거의 같은 답, 높으면 다양 — 사실 조회에는 0 쪽이 안전

API 키가 없다면

로컬 Ollama

구조는 같다

정리

이 1시간에서 꼭 가져갈 것

1. LLM 은 학습 시점까지만 알고, 모르는 것을 지어낼 수 있다
2. 그 한계를 푸는 방법이 RAG — 근거를 찾아 함께 넣어 주는 것
3. LangChain 은 모델이 달라도 같은 코드로 부르게 해 준다
4. prompt | llm | parser 로 부품을 이어 붙인다
5. 대화 기억은 모델이 아니라 우리가 이력을 다시 보내서 만드는 것

7-1 · 이론 1H 마무리

다음은 ChatModel 호출 실습 (1.5H)

첫 호출부터 체인 조립까지 단계별로 만들고, 모델이 모르는 것을 일부러 물어 환각을 확인합니다

- ✓ 가상환경에 langchain 설치, 첫 invoke()
- ✓ 응답 객체와 토큰 사용량 들여다보기
- ✓ ChatPromptTemplate 과 StrOutputParser 로 체인 조립
- ✓ 시스템 메시지·temperature 비교

준비할 것

OpenAI 키 또는 Ollama 모델